Travaux Pratiques — Série 3

Simulation d'une infrastructure Cloud locale

VirtualBox + Ubuntu Server + SSH + VBoxManage + IaC

Pr. A. ABASSI

Faculté des Sciences et Techniques (FST) de Settat

Université Hassan 1er, Settat — Maroc

Série 1 — Simulation d'une infrastructure Cloud locale

Chapitre associé	Chapitre 3 — Virtualisation et infrastructures Cloud (réseaux et stockage virtuels, Infrastructure as Code)
Déroulement	TD : 30 min ; TP : 80 min ; bilan : 10 min
Objectifs	Planifier un adressage IP ; cloner des machines virtuelles ; faire communiquer un serveur Web et un serveur de base de données ; automatiser la création de l'infrastructure par script.
Prérequis	Série 2 terminée : VM `base` avec SSH et instantané `base-propre`.

Partie A — TD (30 min)

Exercice 1 — Plan d'adressage

Le réseau privé `192.168.56.0/24` doit être découpé en quatre sous-réseaux de même taille : Web, base de données, supervision et réserve.

1. Quel masque faut-il utiliser ? Combien d'hôtes utilisables peut-on adresser dans chaque sous-réseau ?
2. Donner l'adresse réseau et l'adresse de diffusion de chaque sous-réseau.
3. Proposer une adresse IP pour la VM `web` et pour la VM `db`.

Exercice 2 — Stockage et Infrastructure as Code

1. Citer deux avantages de décrire une infrastructure dans un script plutôt que de la créer manuellement.
2. Quelle est la différence entre un clone complet et un clone lié d'une machine virtuelle ?

Partie B — TP (80 min)

Étape 1 — Clonage des machines virtuelles

À partir de la machine virtuelle `base`, créer deux clones nommés `web` et `db`.

```
VBoxManage clonevm base --name web --register
VBoxManage clonevm base --name db --register

VBoxManage startvm web --type headless
VBoxManage startvm db --type headless
```

Attention

Avant de démarrer les deux clones simultanément, vérifier que leurs adresses MAC et leurs paramètres réseau ne provoquent pas de conflit. Les noms d'hôte, adresses IP et identifiants de machine doivent également être adaptés.

Configuration de la VM `web`

Ouvrir la console VirtualBox de la VM et exécuter :

```
sudo hostnamectl set-hostname web
```

Si le fichier Netplan de la VM de référence est `/etc/netplan/01-hostonly.yaml` et contient l'adresse `192.168.56.11`, modifier cette adresse :

```
sudo sed -i 's/192.168.56.11/192.168.56.12/' \
/etc/netplan/01-hostonly.yaml

sudo netplan generate
sudo netplan apply
```

Réinitialiser l'identifiant de la machine clonée :

```
sudo rm -f /etc/machine-id
sudo systemd-machine-id-setup
sudo reboot
```

Configuration de la VM `db`

Effectuer les mêmes opérations en utilisant le nom d'hôte `db` et l'adresse IP `192.168.56.13`.

Attention

Le fichier Netplan indiqué est un exemple. Vérifier son nom et son contenu avant d'exécuter les commandes. Si le fichier n'existe pas ou si l'adresse est différente, adapter la configuration. Ces commandes supposent une version d'Ubuntu utilisant Netplan et systemd.

Vérification de la connectivité

Depuis la VM `web`, tester la connexion vers la VM `db` :

```
ping -c 2 192.168.56.13
```

Vérifier également les adresses IP :

```
hostname
ip -br address
```

Étape 2 — Installation du serveur Web (VM `web`)

Installer Nginx :

```
sudo apt-get update
sudo apt-get install -y nginx
```

En environnement hors ligne, utiliser les paquets Debian préalablement téléchargés et leurs dépendances. La commande `dpkg -i *.deb` ne garantit pas à elle seule la résolution des dépendances.

Créer une page Web de test :

```
echo "<h1>Serveur Web - $(hostname)</h1>" \
| sudo tee /var/www/html/index.html

sudo systemctl enable --now nginx
sudo systemctl status nginx --no-pager
```

Depuis l'hôte, tester le serveur si l'adresse IP est accessible :

```
curl http://192.168.56.12
```

Résultat attendu : le contenu HTML de la page créée.

Étape 3 — Installation du serveur de base de données (VM `db`)

Installer PostgreSQL :

```
sudo apt-get update
sudo apt-get install -y postgresql
```

Créer un utilisateur et une base de données :

```
sudo -u postgres psql -c \
"CREATE USER appuser WITH PASSWORD 'ChangeMe123';"

sudo -u postgres psql -c \
"CREATE DATABASE appdb OWNER appuser;"
```

Attention

Le mot de passe `ChangeMe123` est réservé à ce TP. Il ne doit pas être utilisé en production. Un déploiement réel doit employer un secret robuste et éviter d'exposer les mots de passe dans les scripts ou dans Git.

Autoriser les connexions réseau

Repérer les fichiers de configuration PostgreSQL :

```
sudo -u postgres psql -tAc \
"SHOW config_file;"

sudo -u postgres psql -tAc \
"SHOW hba_file;"
```

Dans `postgresql.conf`, configurer l'écoute réseau :

```
listen_addresses = '*'
```

Dans `pg_hba.conf`, ajouter une règle autorisant l'utilisateur `appuser` à accéder à la base `appdb` depuis le réseau du TP :

```
host    appdb    appuser    192.168.56.0/24    scram-sha-256
```

Cette règle suppose que le serveur utilise une version de PostgreSQL compatible avec SCRAM et que le mot de passe de l'utilisateur a été défini avec cette méthode.

Redémarrer le service et vérifier son état :

```
sudo systemctl restart postgresql
sudo systemctl status postgresql --no-pager

sudo ss -ltnp | grep 5432
```

Tester la connexion depuis la VM `web`

Installer le client PostgreSQL :

```
sudo apt-get install -y postgresql-client
```

Tester le port TCP 5432 :

```
nc -zv 192.168.56.13 5432
```

Si la commande `nc` n'est pas installée :

```
sudo apt-get install -y netcat-openbsd
```

Enfin, tester une véritable connexion PostgreSQL :

```
psql -h 192.168.56.13 \
-U appuser \
-d appdb \
-c "SELECT version();"
```

Saisir le mot de passe lorsque le client le demande.

Remarque

Si la connexion échoue, vérifier successivement l'adresse IP, la connectivité réseau, la directive `listen_addresses`, la règle `pg_hba.conf`, le pare-feu et le mot de passe.

Étape 4 — Automatisation : créer l'infrastructure avec un script

Créer un fichier nommé `infra.sh` sur l'hôte disposant de VirtualBox.

Le script ci-dessous vérifie la présence de la VM de référence et de son instantané, crée les clones liés s'ils n'existent pas et démarre les machines qui ne sont pas déjà en cours d'exécution.

```
#!/usr/bin/env bash

# infra.sh : écration des VM du TP Cloud
set -Eeuo pipefail

BASE="base"
SNAPSHOT="base-propre"
NOMS=("web" "db" "monitoring")

# éVrifier l'existence de la VM de éérfrence
if ! VBoxManage showvminfo "$BASE" >/dev/null 2>&1; then
echo "[ERREUR] La VM '$BASE' n'existe pas."
exit 1
fi

# éVrifier l'existence de l'éinstantan
if ! VBoxManage snapshot "$BASE" list \
--machinereadable | grep -Fq "$SNAPSHOT"; then
echo "[ERREUR] éInstantan '$SNAPSHOT' introuvable."
exit 1
fi

for nom in "${NOMS[@]}"; do
if VBoxManage showvminfo "$nom" >/dev/null 2>&1; then
echo "[=] La VM '$nom' existe éàdj."
else
echo "[+] éCration de '$nom'..."
VBoxManage clonevm "$BASE" \
--snapshot "$SNAPSHOT" \
--options link \
--name "$nom" \
--register
fi

# Ne pas éredmarrer une VM éàdj active
if VBoxManage showvminfo "$nom" --machinereadable \
| grep -q '^VMState="running"$'; then
echo "[=] La VM '$nom' est éàdj éédmarre."
else
echo "[+] éDmarrage de '$nom'..."
VBoxManage startvm "$nom" --type headless
```

```
    fi
    done

    echo
    echo "=== Machines virtuelles en cours d'éxcution ==="
    VBoxManage list runningvms
```

Enregistrer le fichier, puis le rendre exécutable :

```
    chmod +x infra.sh
    ./infra.sh
```

Exécuter le script une deuxième fois :

```
    ./infra.sh
```

Questions

1. Que remarquez-vous lors de la deuxième exécution ? Expliquer la notion d'idempotence.
2. Quel est l'avantage d'un clone lié ?
3. Pourquoi la suppression du disque parent peut-elle rendre les clones inutilisables ?
4. Le script configure-t-il les adresses IP et les noms d'hôte ? Quelles modifications faudrait-il ajouter ?

Attention

L'idempotence de ce script concerne principalement la création et le démarrage des VM. Elle ne signifie pas que toute l'infrastructure est automatiquement configurée de manière idempotente. Les adresses IP, les noms d'hôte, les services et les règles de sécurité doivent également être gérés.

Livrable :

1. Le fichier `infra.sh`.
2. Une capture montrant la connexion PostgreSQL depuis `web` vers `db`.
3. Un schéma de l'infrastructure présentant les trois VM, leurs adresses IP et les flux réseau.

Bilan du TP

À la fin de cette séance, vous devez être capable de :

— proposer un plan d'adressage IP ;
— cloner et démarrer des machines virtuelles ;
— configurer la communication entre deux VM ;
— déployer un serveur Web et un serveur PostgreSQL ;
— automatiser partiellement la création d'une infrastructure avec un script Bash et VBoxManage ;
— expliquer les avantages et les limites des clones liés et de l'Infrastructure as Code.